Lập trình mạng · Chương 4

Message broker: RabbitMQ, exchange, queue và binding

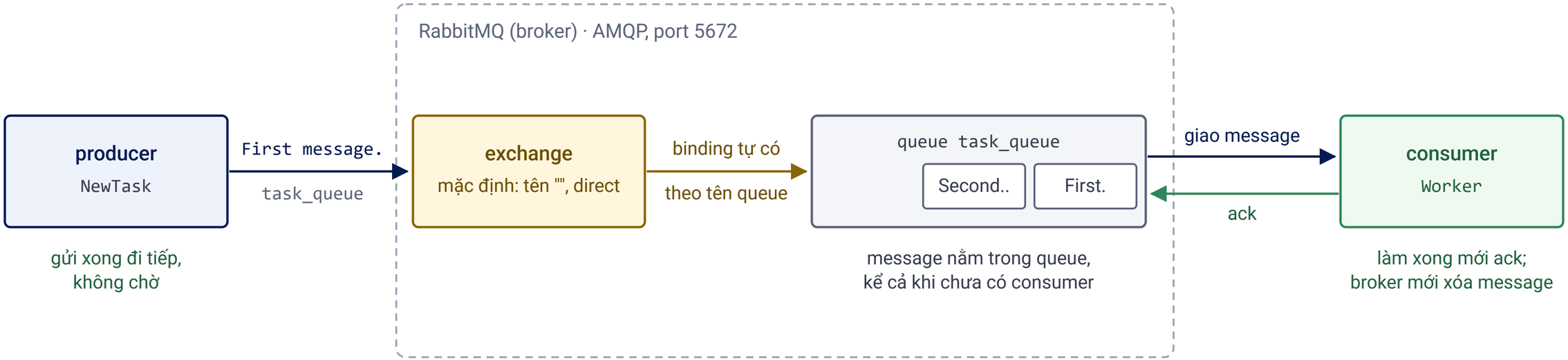
TS. Đặng Ngọc Hùng

Giao việc không cần chờ: giao tiếp bất đồng bộ qua message broker

1. RMI, SOAP, REST, gRPC: bên gọi **chờ** kết quả; bên nhận phải **đang chạy**
2. Việc tốn thời gian như gửi email, xử lý ảnh: bên giao việc không cần chờ, không nên phụ thuộc bên làm
3. **Message broker**: trung gian nhận, lưu và chuyển message, cho phép **giao tiếp bất đồng bộ**

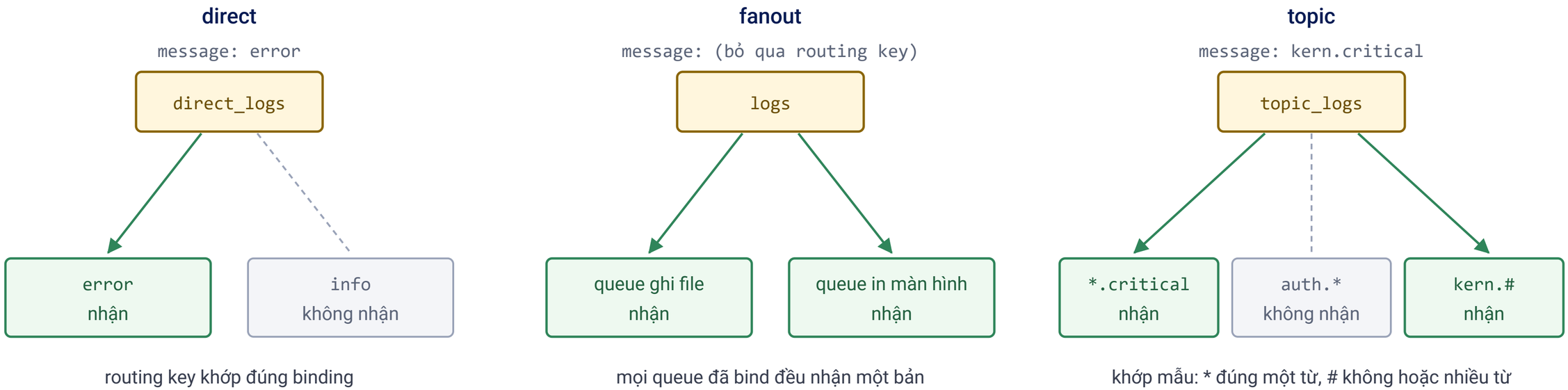
> **Câu hỏi của phần:** message đi qua broker thế nào, và làm sao để message không bị mất?

Producer gửi tới exchange; binding đưa message vào queue; consumer ack



Exchange nhận message, **binding** quyết định **queue** nào nhận; message nằm trong queue đến khi consumer gửi **acknowledgement**.

Ba loại exchange thường dùng: direct, fanout, topic



Loại **exchange** quyết định cách chọn **queue**: direct khớp đúng routing key, fanout gửi mọi queue, topic khớp theo mẫu.

Khai báo queue và gửi việc bằng amqp-client

```java
channel.queueDeclare(TASK_QUEUE_NAME, true, false, false, null);  // durable: queue còn sau khi broker khởi động lại
String message = String.join(" ", argv);
channel.basicPublish("", TASK_QUEUE_NAME,  // exchange mặc định (tên rỗng): routing key là tên queue
        MessageProperties.PERSISTENT_TEXT_PLAIN, message.getBytes(UTF_8));  // message persistent
```

- Producer và worker đều gọi `queueDeclare`: khai báo lại với cùng tham số không đổi gì
- Queue **durable**, message **persistent**: việc còn sau khi broker khởi động lại
- Producer chỉ gọi `basicPublish(exchange, routingKey, ...)`, không biết consumer nào nhận

Worker tắt, việc vẫn chờ trong queue

Java

```
channel.basicQos(1);  // ack thủ công: mỗi lần chỉ nhận một việc chưa ack
doWork(message);
System.out.println(" [x] Done");
channel.basicAck(delivery.getEnvelope().getDeliveryTag(), false);  // làm xong mới ack
channel.basicConsume(NewTask.TASK_QUEUE_NAME, autoAck, deliverCallback, consumerTag -> { });
```

Kết quả chạy

```
$ NewTask First message.
 [x] Sent 'First message.'
$ NewTask Second message..
 [x] Sent 'Second message..'
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
task_queue  2  0
$ Worker --max 2
 [*] Waiting for messages (manual ack)
 [x] Received 'First message.'
 [x] Done
 [x] Received 'Second message..'
 [x] Done
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
task_queue  0  0
```

autoAck: broker xóa message ngay khi giao, worker chết là mất

Java

```
channel.basicConsume(NewTask.TASK_QUEUE_NAME, autoAck, deliverCallback, consumerTag -> { });
Runtime.getRuntime().halt(1);  // tiến trình chết giữa chừng, chưa kịp ack
```

Kết quả chạy

```
$ Worker --auto-ack --crash-on Second message..
 [*] Waiting for messages (autoAck)
 [x] Received 'First message.'
 [x] Done
 [x] Received 'Second message..'
 [!] crash while working on 'Second message..'
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
task_queue  0  0
```

- autoAck: việc thứ nhất xong, **việc thứ hai và thứ ba mất**, queue còn 0

Ack thủ công sau khi làm xong: message chưa ack được giao lại

```java
channel.basicAck(delivery.getEnvelope().getDeliveryTag(), false);  // làm xong mới ack
```

Kết quả chạy

```
$ Worker --crash-on Second message..
 [*] Waiting for messages (manual ack)
 [x] Received 'First message.'
 [x] Done
 [x] Received 'Second message..'
 [!] crash while working on 'Second message..'
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
task_queue  2  0
$ Worker --max 2
 [*] Waiting for messages (manual ack)
 [x] Received 'Second message..' (redelivered)
 [x] Done
 [x] Received 'Third message...'
 [x] Done
```

- Ack thủ công: việc thứ hai quay lại queue, giao lại với cờ **redelivered**; việc thứ ba chưa từng được giao

Câu hỏi 1

Trong RabbitMQ, một message nằm trong queue đến khi nào?

A. Đến khi producer đóng kết nối.

B. Đến khi consumer nhận và gửi ack cho message đó (hoặc được giao với autoAck).

C. Chỉ trong 1 giây, rồi broker tự xóa.

D. Đến khi exchange chuyển message sang queue khác.

Câu hỏi 1

Trong RabbitMQ, một message nằm trong queue đến khi nào?

A. Đến khi producer đóng kết nối.

B. Đến khi consumer nhận và gửi ack cho message đó (hoặc được giao với autoAck).

C. Chỉ trong 1 giây, rồi broker tự xóa.

D. Đến khi exchange chuyển message sang queue khác.

Broker giữ message trong queue cho tới khi consumer ack. Với autoAck, broker coi message là xong ngay khi giao. Producer đóng kết nối không ảnh hưởng message đã gửi.

Câu hỏi 2

Consumer dùng autoAck và bị lỗi, tiến trình chết khi đang xử lý một message. Message đó ra sao?

A. Quay lại queue và được giao cho consumer khác.

B. Được giao lại cho chính consumer đó khi nó chạy lại.

C. Nằm ở trạng thái chưa ack cho tới khi quản trị viên xử lý.

D. Mất, vì broker đã xóa message ngay khi giao.

Câu hỏi 2

Consumer dùng autoAck và bị lỗi, tiến trình chết khi đang xử lý một message. Message đó ra sao?

A. Quay lại queue và được giao cho consumer khác.

B. Được giao lại cho chính consumer đó khi nó chạy lại.

C. Nằm ở trạng thái chưa ack cho tới khi quản trị viên xử lý.

D. Mất, vì broker đã xóa message ngay khi giao.

Với autoAck, broker xóa message ngay khi giao cho consumer. Consumer chết giữa chừng thì message mất; chỉ ack thủ công sau khi xử lý mới giữ được message.

Tổng kết

1. **Message broker** như **RabbitMQ** (**AMQP**) cho **giao tiếp bất đồng bộ**: **producer** gửi rồi đi tiếp

2. **Exchange** nhận message, **binding** chọn **queue**; message chờ trong queue tới khi **consumer** nhận

3. **Acknowledgement** sau khi xử lý xong: consumer chết thì message được giao lại; autoAck làm mất message

Từ khóa cần nhớ giao tiếp bất đồng bộ · message broker · RabbitMQ · AMQP · producer · consumer · exchange · queue · binding · acknowledgement

Mở rộng

Chủ đề nâng cao

- Publisher confirm
- Dead letter exchange
- Headers exchange: định tuyến theo header của message

Bài tập mở rộng

Chạy hai Worker cùng lúc, giao sáu việc dài ngắn khác nhau; so sánh cách broker chia việc khi có và không có basicQos(1).

Tiếp theo

Bài toán ứng dụng: xử lý đơn hàng qua message broker